

Pythonプログラミング 基礎 & 実践

総復習演習編

前回の課題振り返り：全体総評

前回の提出課題、皆さん素晴らしい取り組みでした！自力で考えて動かそうとする姿勢がコードからとてもよく伝わります。

- **Good**： アルゴリズム（FizzBuzzの分岐、最大値、平均、ソート）のコアロジックを多くの人が正しく理解できています。
 - **本日のアプローチ**： 各問題の標準的な解答例を確認しながら、プログラムに潜む細かい改善ポイントをチェックしましょう。
-

【問題1：FizzBuzz】 解答例

3の倍数、5の倍数、15の倍数を正しく判定する基本的な構成です。

```
for i in range(1, 16):  
    if i % 15 == 0:  
        print("FizzBuzz")  
    elif i % 3 == 0:  
        print("Fizz")  
    elif i % 5 == 0:  
        print("Buzz")  
    else:  
        print(i)
```

【問題1】 振り返りと改善ポイント

- **不要なカウントアップ**： `for i in range(...)` のループ内で、`i = i + 1` と手動で加算しているコードがありました。for文は自動で次の値に進むため、手動の加算は不要（バグの原因）です。
-

【問題2：最大値探索】 解答例

暫定1位を決めて、ループで順次比較するアルゴリズムです。

```
numbers = [12, 45, 7, 23, 56, 19]
```

```
# 最初の要素を暫定最大値にする
```

```
max_value = numbers[0]
```

```
for n in numbers:
```

```
    if n > max_value:
```

```
        max_value = n    # 記録更新
```

```
print(max_value)
```

【問題2】 振り返りと改善ポイント

- **初期値の選び方**： `max = 0` からスタートしているコードがいくつかありました。もしリスト内がすべてマイナスの値（例: `[-10, -5, -20]`）だった場合、最大値が `0` と誤判定されてしまいます。初期値は `0` ではなく、**リストの最初の値** (`numbers[0]`) にした方が良いでしょう。
 - **汎用的なループ設計**： `range(0, 6)` のように要素数を「`6`」と固定して比較するのではなく、`for n in numbers:` と書くことで、リストの長さが変わってもそのまま動く汎用性の高いコードになります。
 - **英語の変数名**： `for name in numbers:` のように、数値が入っているループ変数に `name` を使うと、読む人が混乱します。`num` や `n` など適切な名前を使いましょう。
-

【問題3：平均値計算】 解答例

要素の合計値と、個数（リストの長さ）を求めて平均値を計算します。

```
numbers = [12, 45, 7, 23, 56, 19]
```

```
total_sum = 0
```

```
count = 0
```

```
for n in numbers:  
    total_sum += n  
    count += 1
```

```
average = total_sum / count
```

```
print(average)
```

【問題3】 振り返りと改善ポイント

- **予約語（組み込み関数）の上書きに注意：** 変数名に `sum = 0` を使用しているコードがありました。Pythonには最初から `sum()` という合計を計算する便利な関数があります。変数名に `sum` を使うとこの関数が壊れてしまい、以降使えなくなります。変数名は `total_sum` や `total` にしましょう。
 - **計算の位置：** ループの内部で毎回 `Average = total / count` を上書き計算している例がありました。結果は同じになりますが、ループが終わった後に**外側で1回だけ割り算を行う**方がスマートで効率的です。
-

【発展課題：データの選別】 解答例

```
import random

# 1. 乱数を100個生成
data = [random.randint(1, 100) for _ in range(100)]

# 2. 自作バブルソート（昇順に並び替え）
n = len(data)
for i in range(n):
    for j in range(0, n - 1 - i):
        if data[j] > data[j + 1]:
            data[j], data[j + 1] = data[j + 1], data[j] # スワップ

# 3. 上位50個をスライスで抽出 & 4. ランダムに2個抽出
top_50 = data[50:]
result = random.sample(top_50, 2)
print(result)
```

【発展】 振り返り：バブルソートとスワップ

- **バブルソートの比較対象**：バブルソートは、必ず隣り合う2つ (`data[j]` と `data[j+1]`) を比較して入れ替えます。離れた位置 (`data[n]` と `data[m]`) を比較・交換しているコードがありましたが、これはバブルソートではなく選択ソートに近い別のアルゴリズムになってしまいます。
 - Pythonでは `a, b = b, a` と書くだけで `tmp` を使わずに一瞬で入れ替えができます
-

【演習1】 条件付きリスト生成

課題内容

1から50までの整数の中から、「4の倍数」かつ「6の倍数ではない」数だけを抽出し、画面に表示するプログラムを作成してください。

【演習2】 コイン投げシミュレーション

課題内容

「表 (+10点)」か「裏 (-5点)」がランダムに出るコイン投げを20回繰り返してください。

初期の持ち点を100点としたとき、最終的な持ち点を計算して表示するプログラムを作成してください。

【演習3】 段階的割引の団体料金計算関数

課題内容

団体客の合計入場料を計算して返す関数 `calc_group_fee()` を作成してください。料金ルールは以下の通りとします。

- 最初の20人までは通常料金（1,000円/人）
- 21人目から50人目までの人数分については2割引（800円/人）を適用
- 51人目以降の人数分については3割引（700円/人）を適用

※ 関数を作成後、人数が 15、35、60 の場合でそれぞれ呼び出し、合計金額を表示してください。

【演習4】 JSONからの特定データ抽出

課題内容

外部ファイル `employee_data.json` を読み込んでください。

部署（`department`）が "Sales" の社員の名前（`name`）と給与（`salary`）だけをループ処理で画面に表示してください。

【演習5】 JSON形式データの特集条件集計

課題内容

外部ファイル `product_data.json` を読み込んでください。

- 在庫数 (stock) が10個未満の商品の商品名 (name) をすべて表示してください。
- 最後に、それらの該当する商品だけの在庫数の合計を計算して表示してください。

【演習6】 デバッグ（プログラムのバグ修正）

ランダムに生成した10個の点数リスト scores から、60点以上のデータだけを対象に平均点を計算するプログラムですが、意図した通りに動きません。エラーが出ることもあります。

バグのあるプログラム：

```
import random
scores = [random.randint(0, 100) for i in range(10)]
print("生成された点数:", scores)

total = 0
count = 0

for s in scores:
    if s >= 60:
        total += s
        count += 1

average = total / count
print("60点以上の平均点:", average)
```


【演習7】 総合応用（待ち時間シミュレーション）

プログラミング部が文化祭に出展するゲームの待ち時間を計算します。

- 昨年の来訪者の到着時刻： `arrival_times = [0, 3, 4, 10, 11, 12]`
- 1人あたりの体験時間： **3分**
- 全員の中での**最長待ち時間**を求めて表示してください。

※ 1人目の開始は0、終了は3です。

※ 開始時刻は**自身の到着時刻と直前の人の終了時刻のどちらか遅い方（大きい方）**となります。

※ 各自の待ち時間は**自身の開始時刻 - 到着時刻**です。

※ 各自の終了時刻は**自身の開始時刻 + 体験時間**です。

【演習7応用】 待ち時間の最適化とループの中断

「最長待ち時間を10分未満に抑えつつ、体験時間をできるだけ長くしたい」という要望を満たすため、条件に応じたシミュレーションとプログラムの高速化を行います。

- 体験時間 (taiken) を 1 から 15 まで 1 ずつ増やしながらループ（繰り返し）させます。
- 各体験時間において、演習7と同じ到着時刻データ `arrival_times = [0, 3, 4, 10, 11, 12]` を使って最長待ち時間を計算します。
- 最長待ち時間が **10分未満** である場合は、画面に "体験時間 X 分間: 最長待ち時間 Y 分間" と出力してください。
- 最長待ち時間が **10分以上** になった場合は、その体験時間の処理は出力せず、`break` を用いて即座に全体のループを打ち切って（終了して）ください。

【演習8】 総合応用（タスク割り当て）

工芸品の製作タスクを部員3人で分担して製作します。

- 各工芸品の製作日数： `work_days = [4, 1, 3, 1, 3, 4, 2, 4, 3]`
- 部員3人の累積作業日数リスト： `member_days = [0, 0, 0]`（初期値）
- 各工芸品は、その時点で**累積作業日数が最も少ない（最も早く手が空く）部員**が順番に担当します。
- 担当が決まったら、その部員の数値を**現在の数値 + 製作日数**に更新します。

課題：全ての製作が終わる日数（全員の作業が終わる最も遅いタイミング）を表示してください。